Practical 2

1. Binary Search

Introduction

Binary Search is an efficient searching algorithm used to find an element in a sorted array. It works by repeatedly dividing the search interval into two halves until the required element is found or the search interval becomes empty

Aim:

To search for a given element in a sorted list using the Binary Search algorithm.

Objective

- To understand the working of Binary Search.
- To efficiently locate an element in a sorted array.

Algorithm

1. Read the number of elements.
2. Enter the elements in sorted order.
3. Read the search element.
4. Set low = 0 and high = n-1.
5. Find the middle element using mid = (low + high) // 2.
6. If the middle element equals the search element, display its position.
7. If the search element is greater, search the right half.
8. Otherwise, search the left half.
9. Repeat until the element is found or low > high.
10. If not found, display "Element not found."

Pseudocode

START
Read n
Read sorted array A
Read key

```
low = 0
high = n - 1

WHILE low <= high mid
   = (low + high) // 2

   IF A[mid] == key
      Print "Element found at position", mid + 1
      STOP
   ELSE IF A[mid] < key
      low = mid + 1
   ELSE high = mid
   - 1
   ENDIF
ENDWHILE

Print "Element not found"
STOP
```

Code:

```
n = int(input("Enter the number of elements: "))
arr = []

print("Enter the elements in sorted order:")
for i in range(n):
    arr.append(int(input()))

key = int(input("Enter the element to search: "))

low = 0
high = n - 1
found = False

while low <= high:
    mid = (low + high) // 2

    if arr[mid] == key:
        print("Element found at position", mid + 1)
        found = True
        break
    elif arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1

if not found:
    print("Element not found")
```

```
Enter the number of elements: 5
Enter the elements in sorted order:
10
20
30
40
50
Enter the element to search: 30
Element found at position 3
```

Advantages

- Faster than Linear Search.
- Time complexity is O(log n).
- Efficient for large datasets.
- Reduces the number of comparisons.

Disadvantages

- Works only on sorted arrays.
- Sorting the data beforehand may require additional time. □ Not suitable for frequently changing datasets.

Time Complexity

- Best Case: O(1)
- Average Case: O(log n) □ Worst Case: O(log n)

Space Complexity

- O(1) (Iterative implementation)

Conclusion

Binary Search is one of the most efficient searching algorithms for sorted datasets. By repeatedly dividing the search space into halves, it significantly reduces the number of comparisons, making it much faster than Linear Search for large collections of data.

2. Linear Search

Introduction

Linear Search is the simplest searching algorithm. It searches for a target element by checking each element in the list sequentially from the beginning until the element is found or the end of the list is reached. It can be used on both sorted and unsorted arrays.

Aim

To search for a given element in a list using the Linear Search algorithm. Objective

- To understand the concept of sequential searching.
- To locate the position of a specified element in a list.
- To learn the implementation of Linear Search in Python. Algorithm

1. Read the number of elements.
2. Store the elements in an array.
3. Read the element to be searched.
4. Compare the search element with each element of the array one by one.
5. If a match is found, display its position.
6. If the end of the array is reached without finding the element, display "Element not found."

Pseudocode

```
START
Read n
Read array A
Read key

FOR i = 0 to n-1
   IF A[i] == key
      Print "Element found at position", i+1
      STOP
```

ENDIF
ENDFOR

Print "Element not found"
STOP
Code:

1.Linear Search Method

```
2]: n = int(input("Enter the number of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
key = int(input("Enter the element to search: "))
found = False
for i in range(n):
    if arr[i] == key:
        print("Element found at position:", i + 1)
        found = True
        break
if not found:
    print("Element not found")
```

```
Enter the number of elements: 9
Enter the elements:
1
5
6
8
9
7
4
6
2
Enter the element to search: 5
Element found at position: 2
```

Advantages

- Very simple and easy to implement.
- Works with both sorted and unsorted arrays.
- No preprocessing or sorting is required. Suitable for small datasets.

Disadvantages

- Inefficient for large datasets.
- Searches each element one by one.
- Time complexity increases as the size of the data increases.

Time Complexity

- Best Case: O(1)
- Average Case: O(n)
- Worst Case: O(n)

Space Complexity

- O(1)

Conclusion

Linear Search is a basic searching technique that checks each element sequentially until the required element is found. It is easy to understand and implement, making it suitable for small datasets and unsorted lists. However, for large datasets, more efficient algorithms such as Binary Search are preferred.